

- a. Name of GenAI programs = Gemini and ChatGPT
- b. Accessed Gemini from its website (<https://gemini.google.com/app>) and ChatGPT from its windows application.
- c. generate C++ code using objects, not functions, to implement the following requirements
 - The program will prioritize emails for a busy company CEO.
 - You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
 - You must implement a MaxHeap using a list-based implementation. Then use MaxHeap to handle all your email prioritizing for the CEO.
 - You must create functions from scratch. Do not include pre-existing heap modules.

Test File format:

Command	Description
EMAIL <sender category>, <subject line>, <date>	The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows: • Boss – read first • Subordinate – read next • Peer – read next • ImportantPerson – read next • OtherPerson – read last If there is more than one from a sender, then the newest email (not the oldest) should be read first. EMAIL is followed by space. The rest of the fields are delimited. Assume <sender category> is one of the five strings listed above. Assume <subject line> is a string which may contain spaces, but not commas Assume <date> is in the format: MM-DD-YYYY
NEXT	Next email for the CEO to read. Display the information on the terminal in the following format: Sender: Subject: Date:
READ	CEO has read email and dealt with it

COUNT	Display current count of unread emails
-------	--

- The user will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

Sample Test File:

```
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
```

Your program should display this:

```
There are 6 emails to read.

Next email:
    Sender: Boss
    Subject: Never Mind
    Date: 01-03-2025

Next email:
    Sender: Boss
    Subject: Important
    Date: 12-20-2024

There are 4 emails to read.
```

write in C++

d. Gemini

```
#include <iostream>
#include <string>
#include <vector>
#include <sstream>

using namespace std;

// Class to parse and compare dates
class Date {
private:
    int month, day, year;

public:
    Date() : month(0), day(0), year(0) {}
    Date(string dateStr) {
        if (dateStr.length() >= 10) {
            month = stoi(dateStr.substr(0, 2));
            day = stoi(dateStr.substr(3, 2));
            year = stoi(dateStr.substr(6, 4));
        }
    }

    bool operator<(const Date& other) const {
        if (year != other.year) return year < other.year;
        if (month != other.month) return month < other.month;
        return day < other.day;
    }
};

// Class representing an individual Email
class Email {
private:
    string sender;
    string subject;
    string dateStr;
    Date date;
    int priorityScore;

    void determinePriority() {
```

```

        if (sender == "Boss") priorityScore = 5;
        else if (sender == "Subordinate") priorityScore = 4;
        else if (sender == "Peer") priorityScore = 3;
        else if (sender == "ImportantPerson") priorityScore = 2;
        else if (sender == "OtherPerson") priorityScore = 1;
        else priorityScore = 0;
    }

public:
    Email(string s, string sub, string d) : sender(s), subject(sub),
dateStr(d), date(d) {
        determinePriority();
    }

    // Compares priority first. If tied, newer date (greater date)
is read first
    bool operator<(const Email& other) const {
        if (priorityScore != other.priorityScore) {
            return priorityScore < other.priorityScore;
        }
        return date < other.date;
    }

    void print() const {
        cout << "    Sender: " << sender << "\n"
            << "    Subject: " << subject << "\n"
            << "    Date: " << dateStr << "\n\n";
    }
};

// List-based MaxHeap implementation from scratch
class MaxHeap {
private:
    vector<Email> heap;

    int parent(int i) { return (i - 1) / 2; }
    int leftChild(int i) { return (2 * i) + 1; }
    int rightChild(int i) { return (2 * i) + 2; }

    void heapifyUp(int i) {

```

```

        while (i > 0 && heap[parent(i)] < heap[i]) {
            swap(heap[parent(i)], heap[i]);
            i = parent(i);
        }
    }

    void heapifyDown(int i) {
        int maxIndex = i;
        int left = leftChild(i);
        int right = rightChild(i);

        if (left < heap.size() && heap[maxIndex] < heap[left]) {
            maxIndex = left;
        }
        if (right < heap.size() && heap[maxIndex] < heap[right]) {
            maxIndex = right;
        }
        if (i != maxIndex) {
            swap(heap[i], heap[maxIndex]);
            heapifyDown(maxIndex);
        }
    }

public:
    void insert(const Email& email) {
        heap.push_back(email);
        heapifyUp(heap.size() - 1);
    }

    void extractMax() {
        if (heap.empty()) return;
        heap[0] = heap.back();
        heap.pop_back();
        if (!heap.empty()) {
            heapifyDown(0);
        }
    }

    const Email* getMax() const {
        if (heap.empty()) return nullptr;
    }

```

```

        return &heap[0];
    }

    int getSize() const {
        return heap.size();
    }

    bool isEmpty() const {
        return heap.empty();
    }
};

// Class to manage the CEO's inbox operations
class CEOInbox {
private:
    MaxHeap emailQueue;

public:
    void processCommand(const string& commandLine) {
        if (commandLine.substr(0, 5) == "EMAIL") {
            string data = commandLine.substr(6);

            size_t firstComma = data.find(',');
            size_t secondComma = data.find(',', firstComma + 1);

            if (firstComma != string::npos && secondComma !=
string::npos) {
                string sender = data.substr(0, firstComma);
                string subject = data.substr(firstComma + 1,
secondComma - firstComma - 1);
                string dateStr = data.substr(secondComma + 1);

                Email newEmail(sender, subject, dateStr);
                emailQueue.insert(newEmail);
            }
        }
        else if (commandLine == "NEXT") {
            const Email* nextEmail = emailQueue.getMax();
            if (nextEmail != nullptr) {
                cout << "Next email:\n";
            }
        }
    }
};

```

```

        nextEmail->print();
    }
}
else if (commandLine == "READ") {
    emailQueue.extractMax();
}
else if (commandLine == "COUNT") {
    cout << "There are " << emailQueue.getSize() << " emails
to read.\n\n";
}
}
};

int main() {
    CEOInbox inbox;
    string line;

    // Reads from standard input (e.g., a redirected test file)
    while (getline(cin, line)) {
        if (!line.empty() && line.back() == '\\r') {
            line.pop_back();
        }
        if (line.empty()) continue;

        inbox.processCommand(line);
    }

    return 0;
}

```

ChatGPT -

```

#include <iostream>
#include <vector>
#include <string>
#include <sstream>

using namespace std;

class Email {

```

```
private:
    string sender;
    string subject;
    string date;
    int priority;

public:
    Email(string s, string sub, string d) {
        sender = s;
        subject = sub;
        date = d;

        if (sender == "Boss")
            priority = 5;
        else if (sender == "Subordinate")
            priority = 4;
        else if (sender == "Peer")
            priority = 3;
        else if (sender == "ImportantPerson")
            priority = 2;
        else
            priority = 1;
    }

    string getSender() {
        return sender;
    }

    string getSubject() {
        return subject;
    }

    string getDate() {
        return date;
    }

    int getPriority() {
        return priority;
    }
}
```

```

// Converts MM-DD-YYYY into YYYYMMDD
// so dates can be compared numerically.
int getDateValue() {
    int month, day, year;
    char dash1, dash2;

    stringstream ss(date);
    ss >> month >> dash1 >> day >> dash2 >> year;

    return year * 10000 + month * 100 + day;
}

// Returns true if this email has higher priority
// than the other email.
bool hasHigherPriority(Email other) {
    if (priority != other.getPriority())
        return priority > other.getPriority();

    // If same sender category, newer email comes first.
    return getDateValue() > other.getDateValue();
}
};

class MaxHeap {
private:
    vector<Email> heap;

    int parent(int index) {
        return (index - 1) / 2;
    }

    int leftChild(int index) {
        return 2 * index + 1;
    }

    int rightChild(int index) {
        return 2 * index + 2;
    }
}

```

```

void swapEmails(int first, int second) {
    Email temp = heap[first];
    heap[first] = heap[second];
    heap[second] = temp;
}

void heapifyUp(int index) {
    while (index > 0) {
        int p = parent(index);

        if (heap[index].hasHigherPriority(heap[p])) {
            swapEmails(index, p);
            index = p;
        }
        else {
            break;
        }
    }
}

void heapifyDown(int index) {
    int size = heap.size();

    while (true) {
        int left = leftChild(index);
        int right = rightChild(index);
        int largest = index;

        if (left < size &&
            heap[left].hasHigherPriority(heap[largest])) {
            largest = left;
        }

        if (right < size &&
            heap[right].hasHigherPriority(heap[largest])) {
            largest = right;
        }

        if (largest != index) {
            swapEmails(index, largest);
        }
    }
}

```

```

        index = largest;
    }
    else {
        break;
    }
}

}

public:
    void insert(Email email) {
        heap.push_back(email);
        heapifyUp(heap.size() - 1);
    }

    bool isEmpty() {
        return heap.empty();
    }

    int getCount() {
        return heap.size();
    }

    // Displays the highest-priority email without removing it.
    void nextEmail() {
        if (isEmpty())
            return;

        cout << "Next email:" << endl;
        cout << "    Sender: " << heap[0].getSender() << endl;
        cout << "    Subject: " << heap[0].getSubject() << endl;
        cout << "    Date: " << heap[0].getDate() << endl;
        cout << endl;
    }

    // Removes the highest-priority email.
    void readEmail() {
        if (isEmpty())
            return;

        heap[0] = heap[heap.size() - 1];

```

```

        heap.pop_back();

        if (!heap.empty())
            heapifyDown(0);
    }
};

class CEOInbox {
private:
    MaxHeap emailQueue;

public:
    void addEmail(string sender, string subject, string date) {
        Email email(sender, subject, date);
        emailQueue.insert(email);
    }

    void processCommand(string command) {
        if (command == "NEXT") {
            emailQueue.nextEmail();
        }
        else if (command == "READ") {
            emailQueue.readEmail();
        }
        else if (command == "COUNT") {
            cout << "There are "
                 << emailQueue.getCount()
                 << " emails to read." << endl;
            cout << endl;
        }
    }
};

int main() {
    CEOInbox inbox;

    string line;

```

```

while (getline(cin, line)) {

    if (line.substr(0, 6) == "EMAIL ") {
        string emailData = line.substr(6);

        string sender;
        string subject;
        string date;

        stringstream ss(emailData);

        getline(ss, sender, ',');
        getline(ss, subject, ',');
        getline(ss, date);

        inbox.addEmail(sender, subject, date);
    }
    else if (line == "NEXT") {
        inbox.processCommand("NEXT");
    }
    else if (line == "READ") {
        inbox.processCommand("READ");
    }
    else if (line == "COUNT") {
        inbox.processCommand("COUNT");
    }
}

return 0;
}

```

- e. Correctness: Both programs work on normal input, and the heap code is right in both. If two emails have the same sender type and date, the order is random in both. Gemini can use junk values if the date is too short, and it can crash on a bad date or a line that is just EMAIL, but it handles Windows line endings (`\r`). ChatGPT does not crash on bad lines, but it fails on `\r`, so commands like NEXT would do nothing.

Speed: Both are $O(m \log n)$, where n is the number of emails in the heap and m is the number of commands. In practice Gemini is faster, because it reads the date once when the email is made. ChatGPT copies whole emails and re-reads both dates on every comparison. You won't notice on small tests, but you would on huge inputs.

Memory: Both use about the same memory, $O(n \cdot L)$, where L is the length of the text in each email. Gemini stores each date twice, which is a small extra cost. ChatGPT makes many short-lived copies while running, but frees them right away.

Easy to maintain: Gemini has the cleaner design. Its comparison reads simply ($a < b$) and the heap only stores data, but it has an unfinished Date class, and an unused include. ChatGPT is simpler to read, but its heap prints directly (hard to reuse), it copies emails when it doesn't need to, and command reading is split across two places. Both could use more comments.

- f. I would pick Gemini's program as the basis. Its design is cleaner: the comparison is easy to read ($a < b$), and the heap only stores data and does not print anything. It is also faster, because it reads each date one time when the email is made, while ChatGPT copies emails and re-reads dates on every comparison. It handles Windows line endings ($\backslash r$) too, so commands will not silently fail on test files. Its weak spots are easy to fix: set the date numbers to 0 when the date is too short, and remove the unused include.
- g. Correctness: I fixed the places where the program could break. The date now always starts at 0, so it never uses junk values. A bad date or a line with just EMAIL no longer crashes it. Extra spaces and Windows line endings are cleaned up. Emails with the same sender type and date now come out in the order they arrived. It also reads a .txt file now, which the old code could not do. Runs automatically if its called input.txt, other wise shifts to keyboard commands in the terminal. You can also run other .txt files by `./app < [filename].txt`

Space complexity: Memory use for the emails is the same, $O(n \cdot L)$, where n is the number of emails and L is the length of the text in each. I changed one function from recursion to a loop, so it no longer uses extra stack memory. I kept the date text stored twice, because the program needs it to print the date exactly as it was typed.

Execution time: The speed is still $O(m \log n)$, where m is the number of commands and n is the number of emails. The original code was already good here, so a big change was not needed. I only made small gains by passing text by reference and moving emails instead of copying them.

Maintainability: A new trim function cleans each line in one place, and a new run function reads commands from a file or the keyboard. I used `size_t` for heap positions to avoid mixed number types. I did not change the priority if-chain, because it is short and clear. Comments on each line and prologue comments.